

RESTful API & 모델 직렬화

노트북 안에 갇힌 모델을, 누구나 호출할 수 있는 서비스로 꺼내기 위한 첫걸음

학습이 "좋은 모델을 만드는 일"이라면, 배포는 "그 모델을 세상에 꺼내놓는 일"입니다. 오늘은 환경 통제(venv), 소통 규약(REST·HTTP·JSON), 그리고 모델을 파일로 옮기는 직렬화를 다룹니다.

오늘 끝내면 할 수 있는 것

다섯 개 섹션을 관통하는 네 가지 목표입니다.

01

학습과 배포를 구분한다

"내 컴퓨터에서는 되는데" 문제가 왜 생기는지 설명하고, 배포가 풀어야 할 진짜 과제를 정의할 수 있다.

02

실행 환경을 통제한다

`venv`로 의존성을 격리하고 `requirements.txt`로 같은 환경을 동료의 컴퓨터에서 재현할 수 있다.

03

API의 언어로 말한다

REST 원칙·HTTP 메서드(GET/POST)·상태 코드·JSON으로 서버와 주고받는 구조를 읽고 쓸 수 있다.

04

모델을 파일로 옮긴다

`state_dict` · TorchScript · ONNX 세 직렬화 방식의 트레이드오프를 알고 상황에 맞게 고를 수 있다.

여러분이 만든 모델, 지금 어디에 있습니까?

여러분의 모델을, 개발자가 아닌 사람이 지금 당장 사용할 수 있습니까?

대부분의 모델은 노트북에만 살아 있습니다. `model.load_state_dict(...)`

→ `model(input)` 형태로, 입력을 직접 텐서로 만들고 추론을 돌려 결과를 눈으로 확인하는 방식 — 내 노트북 안에서만 동작합니다.

파일 하나만 건네면 공장 막힙니다. 동료에게 `best_model.pth` 를 보내면

`ModuleNotFoundError: transformers`, 이어 `size mismatch for fc.weight` 같은 에러가 줄줄이 납니다.

원인은 환경의 어긋남입니다. 라이브러리 버전이 다르고, 모델 구조 코드가 없고, PyTorch 버전까지 맞지 않습니다. 결국 화면 공유로 한 시간, "그냥 네가 돌려서 결과만 보내줘"로 끝납니다.

문제의 뿌리는 셋입니다. 코드(모델 클래스 정의)가 함께 가지 않고, 의존성(라이브러리·버전)이 공유되지 않으며, 가중치를 만든 런타임(PyTorch 버전)이 불러오는 쪽과 어긋납니다.

가중치 한 장으론 부족합니다. 숫자만 담긴 `.pth` 로는 이 셋 중 어느 것도 풀리지 않습니다. 파일 하나가 아니라 환경·코드·소통 방식을 함께 준비해야 합니다.

결국 모델이 환경에 갇힌 것입니다. 배포는 이 잠금을 푸는 작업이며, 오늘 그 첫 자물쇠부터 엽니다.

모델 파일 하나를 건넬 뿐인데

동료에게 `.pth`를 보내는 순간 벌어지는 일 — 다섯 단계로 따라가 봅시다.

STEP 01

전송

`best_model.pth`를 슬랙으로
보냅니다. "이거 한번 돌려봐."

→ STEP 02

첫 에러

동료가 실행하자마자
`ModuleNotFoundError:`
`transformers`. 필요한
라이브러리가 깔려 있지 않습니다.

→ STEP 03

둘째 에러

설치하고 다시 돌리니 `size`
`mismatch for fc.weight`.
가중치만 있고 **모델 구조**
코드가 없습니다.

→ STEP 04

버전 충돌

구조를 맞춰도 PyTorch 버전이 달라
직렬화 포맷이 어긋납니다. "가끔"
나는 에러라 더 골치 아픕니다.

→ STEP 05

포기

결국 화면 공유로 한 시간. "그냥 네
컴퓨터에서 돌려서 결과만 보내줘..."

학습(Training)과 배포(Serving)는 다른 문제다

지금까지 집중한 영역과 앞으로 배울 영역은 목표 자체가 다릅니다.

지금까지 — Training

좋은 모델 만들기

목표는 정확도

검증셋 accuracy를 0.1%라도 올리는 일이 가장 중요했습니다.

환경은 내 노트북

내가 아는 버전, 내가 만든 입력으로 한 번 돌리면 충분합니다.

실행은 일회성

결과를 한 번 확인하면 목적을 달성합니다. 재현은 부차적입니다.

사용자는 나 자신

전처리·후처리를 머릿속으로 알고 있으니 따로 적지 않습니다.

앞으로 — Serving

그 모델을 세상에 꺼내기

목표는 동작하는 서비스

일단 끊김 없이 돌아가는 것이 정확도보다 먼저입니다.

환경은 남의 컴퓨터

다른 OS·다른 버전·다른 사람에게서도 똑같이 돌아야 합니다.

실행은 반복·상시

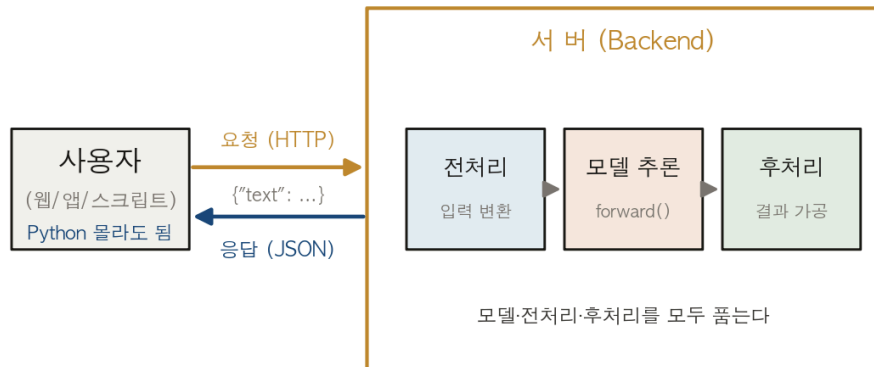
수많은 요청이 24시간 들어옵니다. 재현성이 생명입니다.

사용자는 비개발자

전처리·추론·후처리를 서버가 대신 품고 숨겨야 합니다.

사용자와 모델 사이에 "서버"를 세운다

모델 배포 = 사용자가 **Python을 몰라도** 모델을 쓸 수 있도록, 모델·전처리·후처리를 품은 **서버**를 두고 **HTTP**로 소통하게 만드는 일.



01 사용자는 가볍다

PyTorch도, 모델 구조 코드도, 전처리 로직도 설치할 필요가 없습니다. 무거운 것은 전부 서버에 두고, 사용자는 약속된 형식으로 요청만 보냅니다.

02 서버가 전부 품는다

입력 변환(전처리) → 모델 forward → 결과 가공(후처리)을 서버 안에서 한 번에 처리합니다. 모델 파일과 그 주변 코드가 한 곳에 모입니다.

03 소통은 HTTP·JSON

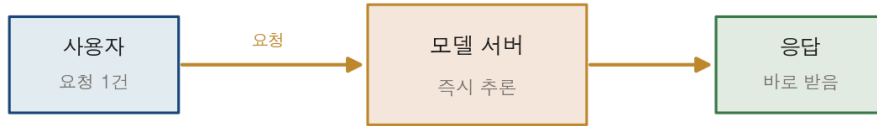
요청과 응답 모두 표준 형식이라, 웹·앱·다른 언어 어디서든 같은 방식으로 호출됩니다. 새 규격을 배울 필요가 없습니다.

04 관심사가 분리된다

모델 내부를 어떻게 바꾸든, 요청·응답 형식만 지키면 사용자 코드는 손댈 필요가 없습니다. 이것이 배포가 주는 가장 큰 자유입니다.

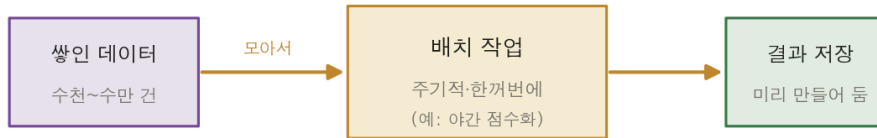
실시간으로 답할까, 모아서 처리할까

ONLINE — 실시간



핵심 = 낮은 지연(latency) · 챗봇·추천 · 오늘의 FastAPI API

BATCH — 일괄



핵심 = 처리량(throughput) · 즉시성 불필요할 때

오늘 만드는 REST API는 'online(실시간)' 서버에 해당한다.

01 Online — 실시간

요청이 오면 **즉시** 추론해 바로 응답합니다. 챗봇·추천처럼 사용자가 결과를 기다리는 경우로, 낮은 **지연(latency)**이 핵심입니다.

02 Batch — 일괄

쌓인 데이터를 주기적으로 한꺼번에 추론합니다. 야간 점수화처럼 즉시성이 필요 없을 때, **처리량(throughput)**이 핵심입니다.

03 무엇을 언제

사용자가 결과를 기다리면 online, 미리 만들어 뒀도 되면 batch. 같은 모델도 쓰임에 따라 두 방식 모두 가능합니다.

04 이 과정의 선택

Day 1~8은 실시간 요청-응답, 즉 **online 서버**를 만듭니다. 그래서 REST API와 빠른 응답이 내내 중심에 있습니다.

추론 서버는 이 세 가지로 평가된다

online 서버에선 **얼마나 빠르고, 안정적으로, 많이** 처리하는지가 곧 품질입니다.

p99

지연 (Latency)

응답까지 걸린 시간. 평균이 50ms라도 **p99**(상위 1% 최악값)가 2초면 100명 중 1명은 답답함을 느낍니다. 그래서 평균이 아니라 p99-p999를 봅니다.

QPS

처리량 (Throughput)

초당 처리한 요청 수(Queries Per Second). 100→200 QPS로 끌어올리면 같은 트래픽에 서버가 절반이면 됩니다. 배치·비동기·경량화가 이를 키웁니다.

99.9%

가용성 (Availability)

장애 없이 응답한 비율. 99.9%라도 한 달에 약 **43분**은 죽는다는 뜻. 무상태로 서버를 여러 대 두고 헬스체크로 죽은 서버를 빼야 올라갑니다.

완벽한 모델보다, 동작하는 서비스가 먼저다

학습 단계의 1순위는 accuracy였지만, 배포 단계의 1순위는 "일단 돌아가는 것"입니다.

성능이 아니라 배포가 목표입니다. 이 과정은 모델 성능을 끌어올리지 않습니다.

이미 만든 좋은 모델을 **세상 밖으로 꺼내는 일**에 집중합니다 — accuracy 0.1%는 잠시 내려놓아도 좋습니다.

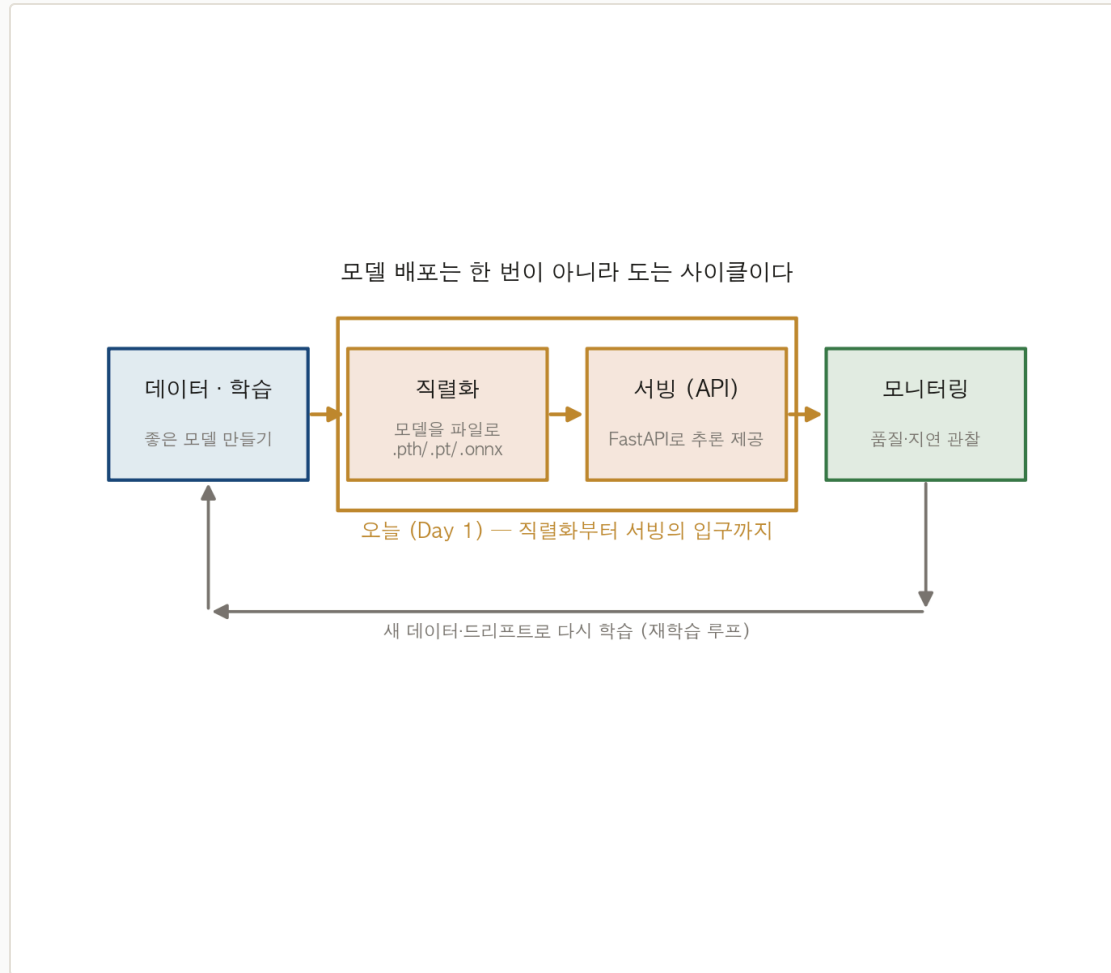
그래서 순서가 중요합니다. 환경 통제(섹션 2) → 소통 규약(섹션 3) → 파일로 옮기기(섹션 4) → 하나의 워크플로로 엮기(섹션 5)의 차례로 갑니다.

순서에는 이유가 있습니다. 환경이 흔들리면 그 위에 무엇을 쌓아도 무너지고, 규약을 모르면 서버를 만들어도 부를 수 없으며, 직렬화가 안 되면 모델은 노트북을 떠날 수 없기 때문입니다.

오늘의 결과물. 화려한 모델이 아니라, **저장 → 불러오기 → 검증**을 마친 모델 파일과 그것을 감쌀 API의 밑그림입니다.

스스로 점검하기. `.pth` 만 건넸을 때 못 쓰는 이유, 학습과 배포의 차이, 사용자와 서버의 소통 방식 — 이 셋을 한 문장씩 답할 수 있으면 섹션 1은 끝입니다.

모델 배포는 한 번이 아니라 '도는 사이클'이다



학습 → 직렬화 → 서빙 → 모니터링 → 재학습. 오늘은 직렬화와 서빙의 입구를 다룬다.

01 학습은 시작일 뿐

좋은 모델을 만드는 일은 전체 사이클의 첫 칸입니다. 그 모델이 세상에서 일하려면 나머지 칸이 필요합니다.

02 직렬화 → 서빙

모델을 파일로 굳히고(직렬화), API로 감싸 요청을 받습니다(서빙). 오늘 Day 1이 바로 이 구간의 **입구**입니다.

03 모니터링

배포 뒤엔 응답 지연·예측 품질을 지켜봐야 합니다. 데이터가 변하면(드리프트) 성능이 **조용히** 떨어집니다.

04 그리고 다시 학습

관찰 결과가 새 학습으로 이어져 사이클이 돕니다. 이것이 MLOps의 골격이며, 이 과정은 그 첫 바퀴를 배웁니다.

가상환경 — 프로젝트마다 독립된 패키지 공간

가상환경(Virtual Environment) = 프로젝트별로 **격리된 패키지 공간**을 만들어 서로 간섭하지 않게 하는 것. Python 3.3+ 에 내장된 `venv` 로 별도 설치 없이 사용합니다.



01 전역 설치의 위험

가상환경 없이 `pip install` 하면 시스템 전역에 깔려, 프로젝트 A의 torch 2.1과 B의 torch 2.3이 충돌합니다.

02 버전 어긋남은 가끔 터진다

대부분은 동작하지만, 직렬화 포맷·내부 API 변경으로 "가끔" 깨지기에 더 위험합니다.

03 venv를 쓰는 이유

Docker와의 호환성, 배포 환경에서의 경량성, 그리고 표준 라이브러리라 어디서든 동일하게 동작합니다.

04 Colab은 예외

이미 격리된 클라우드 환경이므로 venv를 또 만들지 말고 바로 `!pip install` 하면 됩니다.

가상환경 생성부터 비활성화까지

터미널 네 단계면 격리된 Python 환경 하나가 완성됩니다.

STEP 01

생성

`python -m venv .venv` 로
프로젝트 폴더 안에 격리 공간을
만듭니다. `bin·lib·pyvenv.cfg` 가
들어찬 `.venv` 폴더가 생깁니다.



STEP 02

활성화

macOS·Linux는 `source`
`.venv/bin/activate` ,
Windows는
`.venv\Scripts\activate` .
프롬프트 앞에 `(.venv)`
가 붙습니다.



STEP 03

설치

활성화 상태에서 `pip install`
하면 전역이 아니라 `.venv` 안에만
패키지가 들어갑니다. `which`
`python` 으로 경로를 확인합니다.



STEP 04

기록

`pip freeze >`
`requirements.txt` 로 설치된
버전을 목록으로 남깁니다. `.venv`
폴더 자체는 공유하지 않습니다.



STEP 05

비활성화

`deactivate` 한 줄이면 원래 셸로
돌아옵니다. `(.venv)` 표시가
사라지면 격리에서 빠져나온 것입니다.

가상환경은 폴더 하나에 담긴다

`venv` 를 만들면 `.venv` 폴더 안에 격리된 Python 한 벌이 통째로 들어갑니다.

```
.venv/
├─ bin/          # (Win: Scripts/) python·pip, activate 스크립트
├─ include/      # C 확장 헤더 파일
├─ lib/          # 설치된 패키지가 쌓이는 곳
└─ pyvenv.cfg    # 어느 시스템 Python을 기반으로 하는지

# 주피터에서 이 환경을 '커널'로 등록
pip install ipykernel
python -m ipykernel install --user \
    --name myenv --display-name "Model Serving"
```

01 bin/ · Scripts/

activate 스크립트와 격리된 `python` · `pip` 이 여기 있습니다. 활성화하면 이 python이 우선됩니다.

02 lib/

`pip install` 이 패키지를 쌓는 곳. 전역이 아니라 여기에만 설치되어 **격리**가 성립합니다.

03 pyvenv.cfg

이 가상환경이 어느 시스템 Python을 가리키는지 기록한 설정 파일입니다.

04 주피터 커널 등록

노트북에선 `source activate` 가 안 통합니다. `ipykernel` 로 등록한 뒤 Kernel>Change Kernel로 전환합니다.

환경을 "기록"해서 공유한다

.venv 폴더는 용량이 크고 OS에 종속적이라 통째로 보낼 수 없습니다.

대신 어떤 패키지가 어떤 버전으로 필요한지 **목록**을 공유합니다.

직접 작성한 `requirements.txt` 는 사람이 관리하는 핵심 패키지 목록입니다. 읽기 쉽고, 무엇이 왜 필요한지 드러냅니다.

`pip freeze` 로 뽑은 목록은 하위 의존성까지 전부 고정해 정밀 재현에 유리하지만 길어서 읽기 어렵습니다. 실무에서는 둘을 병행합니다.

새 가상환경을 하나 더 만들어 `pip install -r requirements.txt` 로 동료의 컴퓨터를 흉내 내 보면, 같은 버전이 깔리는지 바로 확인할 수 있습니다.

이것만으로 "내 컴퓨터에서는 되는데" 문제의 절반이 풀립니다.



나머지 절반 — OS·시스템 라이브러리까지의 재현은 이후 Docker가 맡습니다.

**"requirements.txt가 환경의 절반을,
Docker가 나머지 절반을 책임진다."**

이 과정의 requirements.txt

역할별로 핵심 패키지만 추렸습니다. 직접 작성한 목록은 무엇이 왜 필요한지가 드러납니다.

```
# 모델
torch
torchvision
# 백엔드 (API)
fastapi
uvicorn[standard]
pydantic
# 프론트엔드
streamlit
# 직렬화 · 추론
onnx
onnxruntime
# 호출 · 유틸
requests
pillow
```

01 모델

`torch` · `torchvision` — 학습과 추론의 토대. 버전에 가장 민감해 보통 `==` 로 고정합니다.

02 백엔드

`fastapi` (웹 프레임워크) · `uvicorn` (실행 서버) · `pydantic` (검증). Day 2의 주역입니다.

03 직렬화·추론

`onnx` · `onnxruntime` — 오늘 만든 모델을 변환하고 실행하는 데 씁니다.

04 재현 테스트

새 venv에서 `pip install -r requirements.txt` → 같은 버전이 깔리면 **환경 재현 성공**입니다.

torch==2.12.0, 이 숫자들의 약속

버전을 `==`로 고정하는 이유는 시맨틱 버저닝(SemVer)에 있습니다 — 숫자 세 자리가 호환성을 약속합니다.

2

MAJOR — 깨지는 변경

올라가면 기존 코드가 **멈출 수 있습니다**. 함수 시그니처·직렬화 포맷·기본 동작이 바뀌는 자리라, 올릴 땐 변경 내역을 꼭 확인합니다.

12

MINOR — 기능 추가

호환되는 새 기능이 더해집니다. 새 인자가 생겨도 기본값이 있어 기존 호출은 그대로 — 대개 안심하고 올려도 됩니다.

0

PATCH — 버그 수정

동작은 그대로, 버그·보안만 고칩니다. 거의 항상 올리는 게 이득이라 고정해도 안전한 자리입니다.

8일 내내 쓸 폴더 구조를 먼저 잡는다

역할별로 폴더를 나눠 두면, 모델·백엔드·프론트엔드가 **어디에 사는지**가 구조 자체로 드러납니다.

```
model-serving-course/
├── .venv/           # 가상환경 (Git 제외)
├── models/          # 직렬화된 모델 .pth/.pt/.onnx
├── app/             # FastAPI 백엔드
├── frontend/        # Streamlit 프론트엔드
├── notebooks/       # 학습용 노트북
├── data/            # 샘플 데이터
├── tests/           # 테스트
├── requirements.txt # 의존성 목록
└── .gitignore       # Git 제외 설정
```

01 관심사 분리

`app/` (백엔드)와 `frontend/` (프론트)를 폴더로 갈라 두면, REST의 클라이언트-서버 분리가 디렉토리에 그대로 나타납니다.

02 .gitignore의 역할

`.venv/` · `__pycache__/` · `models/*.pth` 처럼 용량이 크거나 환경 종속적인 것은 Git에 올리지 않습니다.

03 models/ 는 산출물 자리

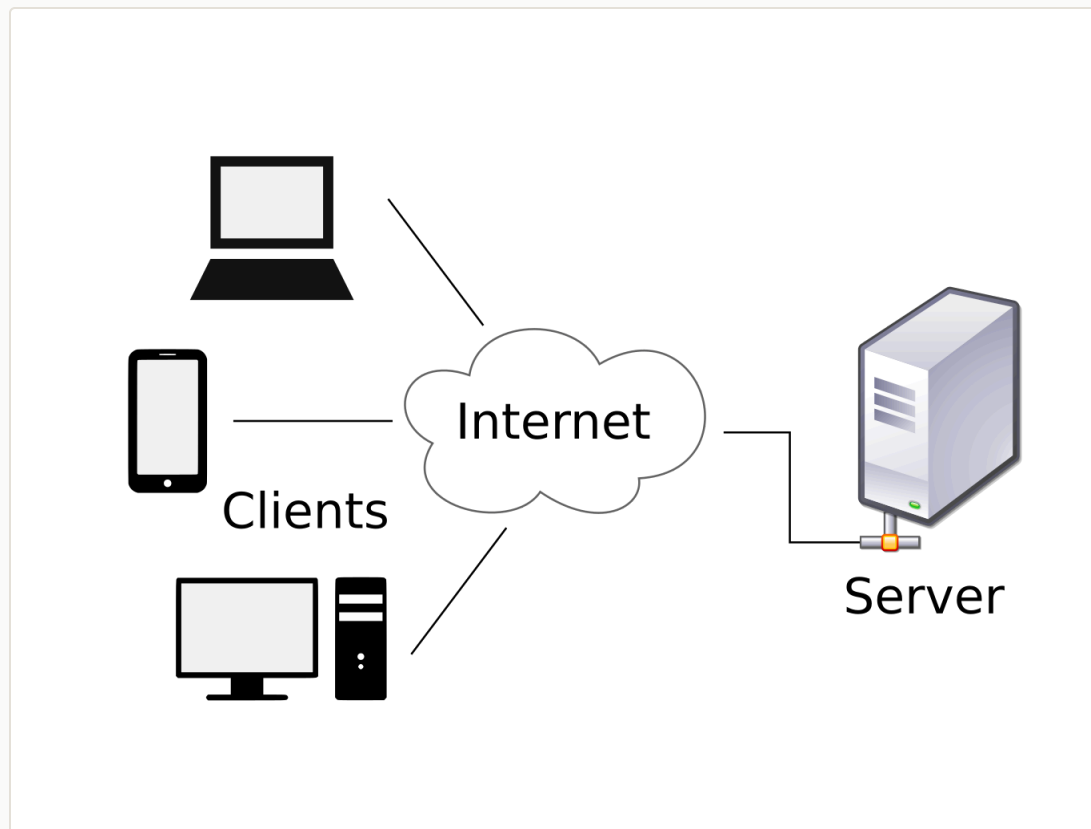
오늘 저장할 `state_dict-TorchScript-ONNX` 파일이 여기 쌓입니다. 코드가 아니라 **결과물**이라 버전 관리에서 뺍니다.

04 오늘 채울 곳

섹션 5 실습에서 `notebooks/`로 학습하고 `models/`에 저장합니다. 나머지 폴더는 Day 2부터 차례로 채워집니다.

API — 약속된 형식으로 주고받는 소통 규약

API(Application Programming Interface) = 약속된 형식으로 요청하면 약속된 형식으로 응답을 받는 소통 규약. 식당의 주문서와 같습니다 — 손님은 주방에 들어가지 않습니다.



01 손님은 주방을 모른다

주문서(API 규약)에 맞춰 요청하면 결과가 나옵니다. 어떤 재료를 어떤 순서로 조리하는지 손님은 알 필요도, 알 수도 없습니다. 이렇게 **내부를 감추는 것이** API의 핵심 가치입니다.

02 모델도 똑같다

사용자는 PyTorch나 GPU 없이도, 정해진 형식으로 데이터를 보내면 서버가 전처리·추론·후처리를 거쳐 결과만 돌려줍니다.

03 클라이언트-서버 분리

UI(프론트엔드)와 모델(백엔드)을 따로 개발·배포합니다. 한쪽 내부가 바뀌어도 **규약만 지키면** 다른 쪽은 멀쩡합니다 — FastAPI와 Streamlit의 관계가 그렇습니다.

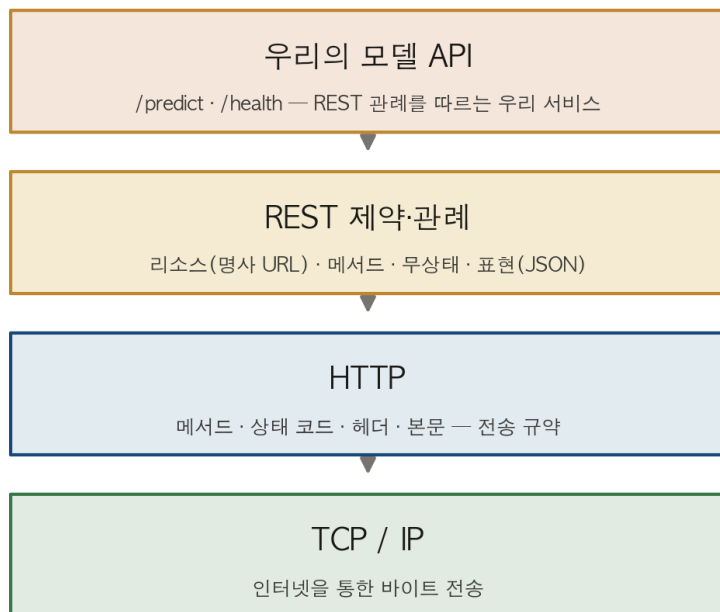
04 언어에 묶이지 않는다

표현을 JSON으로 주고받으므로 Python·JavaScript·Java·Go 어떤 언어로든 호출됩니다. 그래서 API 하나로 웹·앱·다른 서버를 동시에 상대할 수 있습니다.

REST는 '표준'이 아니라 '아키텍처 스타일'이다

REST(REpresentational State Transfer)는 2000년 Roy Fielding이 박사 논문에서 제시한 분산 시스템 **설계 양식**입니다. 지켜야 할 규격이 아니라, 따르면 좋은 **제약들의 모음**입니다.

REST는 표준이 아니라, HTTP 위에 쌓은 "양식"이다



01 표준이 아니라 '정도'의 문제

그래서 "RESTful하다"고 말합니다. 통과·실패를 가르는 인증이 아니라, 제약을 얼마나 지켰는지의 **정도**입니다. Fielding이 제시한 여섯 제약 — 클라이언트-서버, 무상태, 캐시, 균일 인터페이스, 계층 구조, 코드 온 디맨드 — 을 따를수록 더 RESTful합니다.

02 HTTP를 새로 만들지 않는다

이미 있는 HTTP의 메서드·상태 코드·URL·헤더를 **약속된 방식으로 쓰자**는 합의일 뿐입니다. 그래서 별도 규격이나 전용 라이브러리 없이, 브라우저든 서버든 웹이 닿는 어디서나 그대로 통합니다.

03 다른 선택지도 있다

함수 호출처럼 부르는 **RPC·gRPC**, 클라이언트가 필요한 필드만 골라 받는 **GraphQL**. 각자 강점이 다릅니다 — gRPC는 빠른 내부 통신에, GraphQL은 복잡한 데이터 조회에 강합니다.

04 그럼에도 REST가 기본인 이유

리소스 중심이라 직관적이고, 무상태·캐시 친화라 확장이 쉬우며, HTTP만으로 되는 단순함까지 갖췄습니다. 그래서 공개 API의 사실상 기본이 되었습니다.

RESTful API를 지탱하는 다섯 가지 약속

URI는 무엇을(명사) 가리키고, HTTP 메서드는 어떻게(동사) 를 나타냅니다.

① 리소스와 URI — API가 다루는 모든 대상은 리소스이고, 저마다 고유한 URI로 식별됩니다. 자원을 식별하는 이름이 URI이고, 위치(`http://...`)까지 담은 URL은 그중 한 종류 — 웹 API에선 사실상 URL을 씁니다.

`/models` · `/predictions/42` 처럼 명사로 짓고, `/getModels` 같은 동사형은 피합니다.

② 메서드로 행동 표현 — 같은 `/models/v1` 이라도 GET은 조회, PUT은 수정, DELETE는 삭제, POST는 예측 요청. URI가 무엇을, 메서드가 어떻게를 말합니다.

③ 무상태(Stateless) — 서버는 클라이언트 상태를 기억하지 않고, 매 요청이 필요한 정보를 스스로 담습니다. 덕분에 어느 서버가 받아도 같은 결과라 확장(scale-out)이 쉽습니다.

④ 표현으로 전송 — GPU 위의 텐서가 아니라 그 표현(주로 JSON)을 주고받습니다. 그래서 클라이언트는 서버 내부 구조를 몰라도 됩니다.

⑤ 클라이언트-서버 분리 — UI(프론트)와 모델(백엔드)을 독립적으로 개발·배포합니다. 이 과정의 Streamlit과 FastAPI가 바로 그 관계입니다.

좋은 엔드포인트, 나쁜 엔드포인트

URL은 명사로 무엇을, 메서드가 동사로 어떻게. 이 원칙을 지켰는지가 가릅니다.

이렇게 (권장)

리소스 중심

명사로 리소스 표현

`GET /models` · `GET /models/sentiment-v1` — 무엇을 다루는지가 드러납니다.

동작은 메서드가

`POST /models/v1/predict` — 동작은 URL이 아니라 HTTP 메서드가 말합니다.

일관된 복수형

`/models` · `/predictions` 처럼 규칙을 통일하면 다음 엔드포인트가 예측됩니다.

이렇게는 말고

동사·즉흥 작명

URL에 동사

`GET /getModels` · `POST /doPredict` — 동작을 URL에 넣지 않습니다.

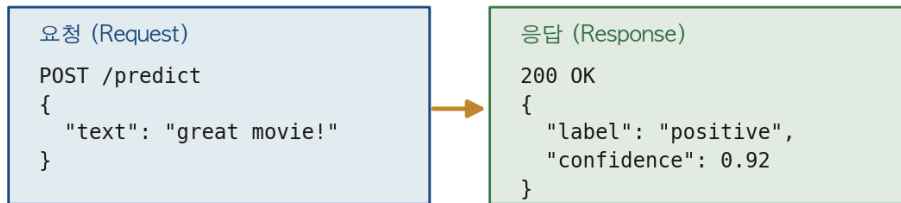
일관성 없는 네이밍

`/model_list_all` 같은 즉흥 작명은 협업과 유지보수를 어렵게 합니다.

메서드 오용

조회에 POST, 생성에 GET처럼 의미와 어긋나게 쓰지 않습니다.

GET과 POST — 모델 배포의 두 동사



HTTP 메서드 — “어떻게” 하고 싶은지

GET

데이터 조회 · 본문 없음 · 멍등

POST

데이터 전송 후 처리 요청 · 추론은 POST

01 GET — 조회

서버에서 정보를 읽어올 때 씁니다. 본문이 없고 필요한 정보는 URL에 담으며, 같은 요청을 여러 번 보내도 결과가 같습니다(멥등성).

02 POST — 처리 요청

데이터를 본문에 담아 보내 처리를 요청합니다. **모델 추론은 대부분 POST** 입니다 — 입력 데이터를 본문에 실어야 하기 때문입니다.

03 요청 본문의 형태

`{"text": "..."}` 처럼 JSON으로 입력을 담습니다. 서버는 이를 파싱해 전처리 → 추론으로 넘깁니다.

04 응답의 형태

`{"label": ..., "confidence": ...}` 같은 JSON과 상태 코드(200)를 함께 돌려줍니다. 이 과정에서는 GET·POST 둘만으로 충분합니다.

요청 본문(JSON)을 담아 보내고, 상태 코드와 함께 JSON 응답을 받는다.

GET·POST 너머 — 안전한 메서드와 멍등성

메서드는 단순한 이름이 아니라 서버에 주는 **약속**입니다. 두 성질 — 안전(safe)과 멍등(idempotent) — 이 그 약속의 핵심입니다.

다섯 메서드 — 모델 배포엔 GET·POST면 충분하지만 나머지도 뜻이 정해져 있습니다. PUT은 전체 수정, PATCH는 부분 수정, DELETE는 삭제입니다.

안전한(safe) 메서드 — 서버 상태를 바꾸지 않는 읽기 전용입니다. GET이 대표적이며, 몇 번을 불러도 데이터가 변하지 않습니다.

멍등한(idempotent) 메서드 — 같은 요청을 여러 번 보내도 결과가 한 번 보낸 것과 같습니다. GET·PUT·DELETE는 멍등이지만, **POST는 멍등이 아닙니다** — 두 번 부르면 두 개가 생길 수 있습니다.

안전한 메서드는 캐시도 쉽습니다. GET처럼 부수효과가 없는 요청은 브라우저·중간 서버가 응답을 **캐싱**해 둘 수 있습니다. 조회를 POST로 만들면 이런 HTTP 기본 이점을 스스로 포기하는 셈입니다.

그래서 재시도가 걸립니다. 네트워크 오류로 다시 보낼 때 멍등한 메서드는 안전하지만, POST 재시도는 중복 생성을 부를 수 있습니다. 추론을 POST로 둘 땐 이 점을 염두에 두어야 합니다.

서버의 대답을 세 자리 숫자로 읽기

HTTP 상태 코드 — 서버의 대답

2xx

성공
요청이 정상 처리됨 · 200 OK

4xx

클라이언트 잘못
400 Bad Request · 422 검증 실패 · 404

5xx

서버 잘못
500 Internal Server Error · 503

01 2xx 성공

요청이 정상 처리됐습니다. 추론이 성공하면 200 OK를 돌려줍니다.

02 4xx 클라이언트 잘못

요청 자체가 잘못된 경우입니다. 400 Bad Request, 입력 검증 실패 시 422, 경로 없음은 404.

03 5xx 서버 잘못

서버 내부에서 에러가 났습니다. 500 Internal Server Error, 모델 미로드 등 일시적 불가는 503.

04 왜 구분이 중요한가

모든 에러를 500으로 뭉뚱그리면 클라이언트가 "내 잘못인지 서버 잘못인지" 알 수 없습니다. 적절한 코드 반환이 좋은 API의 기본기입니다.

첫 자리가 의미를 결정한다 — 2xx 성공, 4xx 클라이언트 잘못, 5xx 서버 잘못.

데이터를 주고받는 공용어, JSON

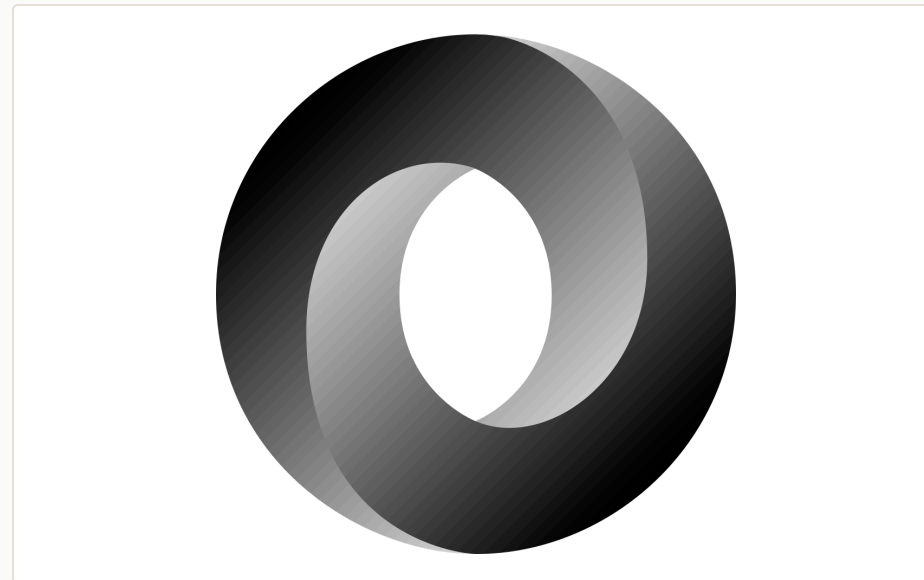
JSON(JavaScript Object Notation)은 API에서 가장 널리 쓰이는 데이터 형식입니다. Python의 딕셔너리와 거의 똑같이 생겼습니다.

키-값 쌍으로 이루어지고 중첩이 가능합니다. 문자열·숫자·불리언·null·배열·객체를 표현할 수 있어, 추론 요청과 응답을 담기에 충분합니다.

`requests.post(url, json=data)`의 `json=`은 파이썬 dict를 JSON 문자열로 직렬화하고 `Content-Type: application/json` 헤더까지 자동으로 붙여 줍니다.

한글을 다룰 때는 `json.dumps(..., ensure_ascii=False)`를 씁니다. 빼면 한글이 `\uXXXX` 유니코드 이스케이프로 깨져 보입니다.

Python의 `True`는 JSON에서 소문자 `true`로, `None`은 `null`로 바뀐다는 점도 기억해 두세요.



표준 텍스트 형식이라 언어·플랫폼에 독립적이다.

"서버 내부가 무엇이든,
바깥세상과는 JSON으로 말한다."

requests로 진짜 API를 불러 본다

개념을 손으로 확인합니다. 파이썬 스크립트로 요청을 보내고 응답·상태 코드를 받아봅니다.

```
import requests

# GET — 정보 조회 (조건은 URL 뒤에)
r = requests.get("https://httpbin.org/get",
                 params={"q": "hello"})
print(r.status_code)      # 200
print(r.json())           # 응답을 dict로

# POST — 데이터 전송 (본문에 JSON)
payload = {"text": "이 영화 재밌다"}
r = requests.post("https://httpbin.org/post", json=payload)
print(r.json()["json"])    # 보낸 본문이 그대로 돌아온다
```

01 status_code

서버의 대답 번호. 200이면 성공, 4xx면 내 요청 문제, 5xx면 서버 문제입니다.

02 json= 파라미터

dict를 JSON 문자열로 바꾸고 `Content-Type` 헤더까지 자동으로 붙여 줍니다.

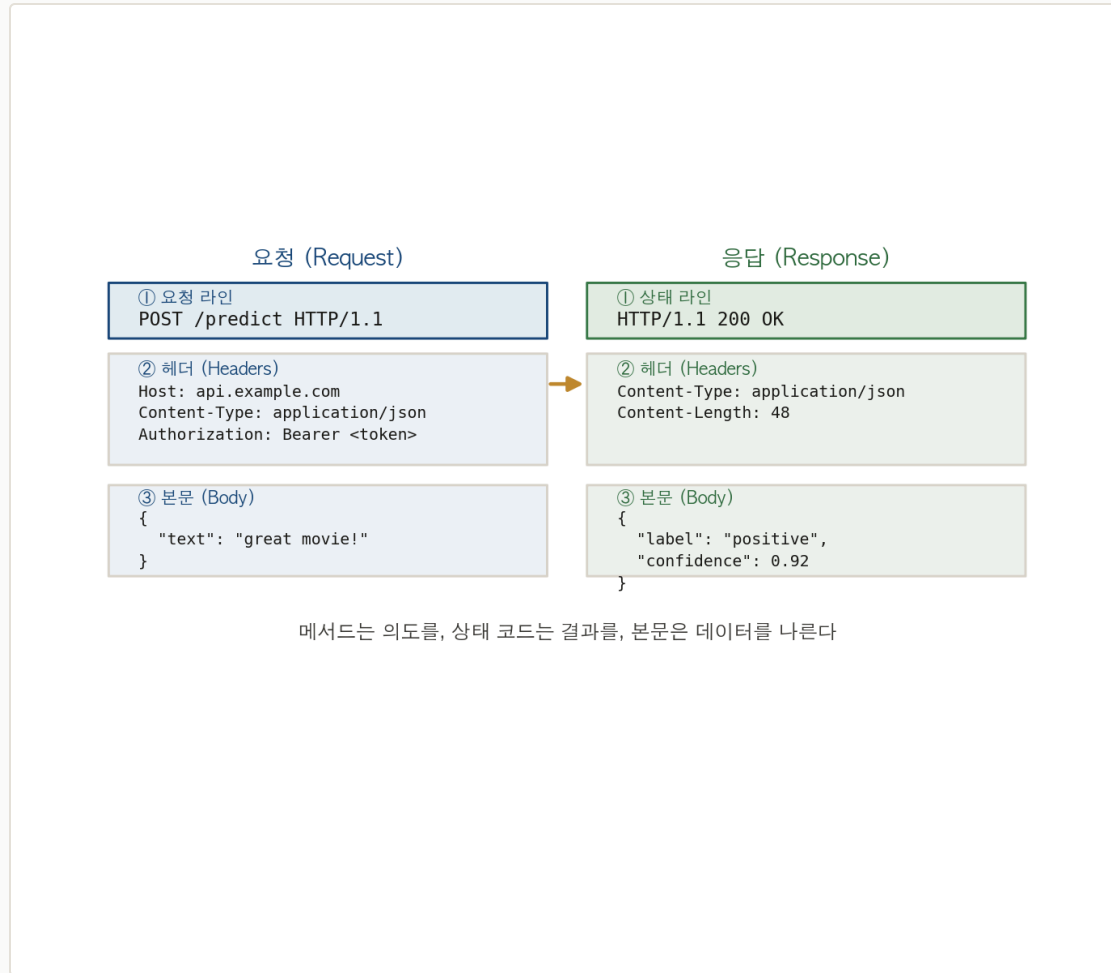
03 .json()

응답 본문(JSON 문자열)을 파이썬 dict로 되돌립니다 — 역직렬화입니다.

04 왜 httpbin

보낸 요청을 그대로 돌려주는 연습용 공개 API라, 무엇을 보냈는지 눈으로 확인됩니다.

요청과 응답은 세 부분으로 이뤄진다



요청 라인/상태 라인 · 헤더 · 본문 — 모든 HTTP 메시지의 공통 뼈대.

01 요청 라인 · 상태 라인

요청은 메서드+경로+버전(`POST /predict HTTP/1.1`), 응답은 버전+상태 코드(`HTTP/1.1 200 OK`)로 시작합니다.

02 헤더 (Headers)

메시지에 대한 메타데이터입니다. `Content-Type` 으로 본문 형식을, `Authorization` 으로 인증 정보를 키:값으로 씁니다.

03 본문 (Body)

실제 데이터입니다. 추론 입력 JSON이 여기 담깁니다. GET처럼 본문이 비는 요청도 있습니다.

04 왜 알아야 하나

FastAPI가 이 구조를 자동 처리하지만, 디버깅 땀 **헤더가 빠졌나, 본문 형식이 틀렸나**를 이 세 부분으로 짚어야 합니다.

코드로 미리 보는 추론 API

오늘 배운 개념이 실제 코드 어디에 들어가는지 — 지금은 구조만 눈에 익혀 두세요.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class PredictRequest(BaseModel):
    text: str

class PredictResponse(BaseModel):
    label: str
    confidence: float

@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    # 1. 전처리 2. 모델 추론 3. 후처리
    return PredictResponse(label="긍정", confidence=0.92)
```

01 HTTP 메서드

`@app.post("/predict")` — 입력을 본문에 담아야 하므로 POST를 씁니다.

02 JSON ↔ 스키마

`PredictRequest` / `PredictResponse` (Pydantic)가 JSON과 파이썬 객체를 자동 변환합니다.

03 상태 코드

성공 200, 입력 오류 422, 서버 에러 500 — FastAPI가 상당 부분 자동 처리합니다.

04 URL 설계

`/predict` — 무엇을 하는 API인지 경로만 봐도 직관적으로 드러납니다.

왜 FastAPI로 서버를 만드나

Day 2에서 본격적으로 다룹니다 — 오늘은 왜 **이걸 고르는지**만 짚고 넘어갑니다.

전통적 선택

Flask · Django

자료가 많고 범용적

오래된 만큼 생태계가 크고 예제가 풍부합니다. 일반 웹사이트·관리자 페이지에 강합니다.

검증·문서는 수동

입력이 올바른지 일일이 `if` 로 확인하고, API 문서도 따로 작성·관리해야 하는 경우가 많습니다.

기본은 동기

Flask는 기본이 동기라, 느린 추론 한 건이 다른 요청을 막기 쉽습니다(별도 설정 필요).

이 과정의 선택

FastAPI

타입 기반 자동 검증

파이썬 타입힌트와 Pydantic으로, 잘못된 입력은 **코드에 닿기 전에** 422로 걸러집니다.

문서가 공짜

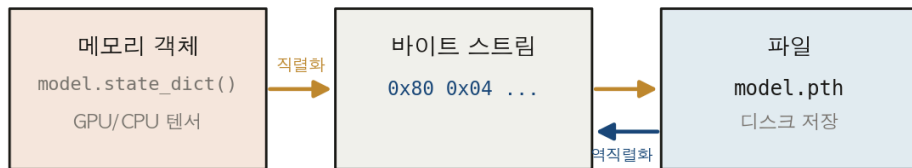
코드의 타입·스키마에서 Swagger UI 문서가 자동 생성돼, 브라우저에서 바로 테스트까지 됩니다.

비동기·빠른 속도

`async` 를 기본 지원하고 Starlette·Pydantic 위에서 동작해, I/O 대기가 많은 추론 서버에 잘 맞습니다.

직렬화 — 메모리 위의 모델을 파일로

직렬화(Serialization) = 메모리에 있는 객체를 **파일로 저장 가능한 형태(바이트)로 변환**하는 것. `torch.save` 가 직렬화, `torch.load` 가 역직렬화입니다.



01 이미 쓰고 있었다

`torch.save(model.state_dict(), "m.pth")` 가 바로 직렬화입니다. 새로운 개념이 아니라 늘 하던 일입니다.

02 학습용 vs 배포용

학습 땐 "내가 다시 불러오면 끝"이지만, 배포 땐 다른 환경·다른 사람·다른 언어가 불러와야 합니다.

03 그래서 선택지가 생긴다

배포를 고려하면 저장 방식의 폭이 넓어지고, 각 방식의 트레이드오프를 이해해야 합니다.

04 오늘 다룰 세 가지

`state_dict`(Pickle 기반) · TorchScript · ONNX — 다음 슬라이드에서 차이를 비교합니다.

state_dict 와 TorchScript — 같은 PyTorch, 다른 그릇

둘 다 PyTorch로 저장하지만, 무엇을 담느냐가 다릅니다.

가중치만

state_dict (.pth)

무엇을 담나

레이어별 가중치 숫자(텐서)만 담습니다. **모델 구조는 빠져** 있습니다.

불러올 때

같은 `SimpleClassifier` 클래스 정의가 **반드시** 있어야 가중치를 끼워 넣을 수 있습니다.

어디에 좋나

가장 가볍고, optimizer 상태까지 곁들이면 학습 재개에 좋습니다. **이 과정의 기본값**입니다.

구조까지

TorchScript (.pt)

무엇을 담나

가중치에 더해 **모델 구조(그래프)**까지 한 파일에 함께 담습니다.

불러올 때

클래스 정의 없이 `torch.jit.load` 한 줄로 끝. 코드 import가 필요 없습니다.

어디에 좋나

Python 없이 C++(libtorch)에서도 실행됩니다. 모델 파일만 따로 배포할 때 강합니다.

PyTorch 계열과 ONNX, 무엇이 다른가



PYTORCH NATIVE

state_dict · TorchScript

필요 런타임

PyTorch / libtorch가 깔려 있어야 로드됩니다. Python 생태계의 표준 저장 방식입니다.

강점

파이썬 객체 그대로라 `print` 디버거로 들여다보기 쉽습니다. 코드와 모델을 함께 다루는 개발 단계에 최적입니다.

확장자

`.pth` (state_dict, 가중치만) · `.pt` (TorchScript, 구조 포함). 둘 다 PyTorch 런타임에서 로드합니다.



OPEN STANDARD

ONNX

필요 런타임

ONNX Runtime만 있으면 PyTorch 없이도 실행됩니다. 프레임워크·언어에 독립적인 개방형 포맷입니다.

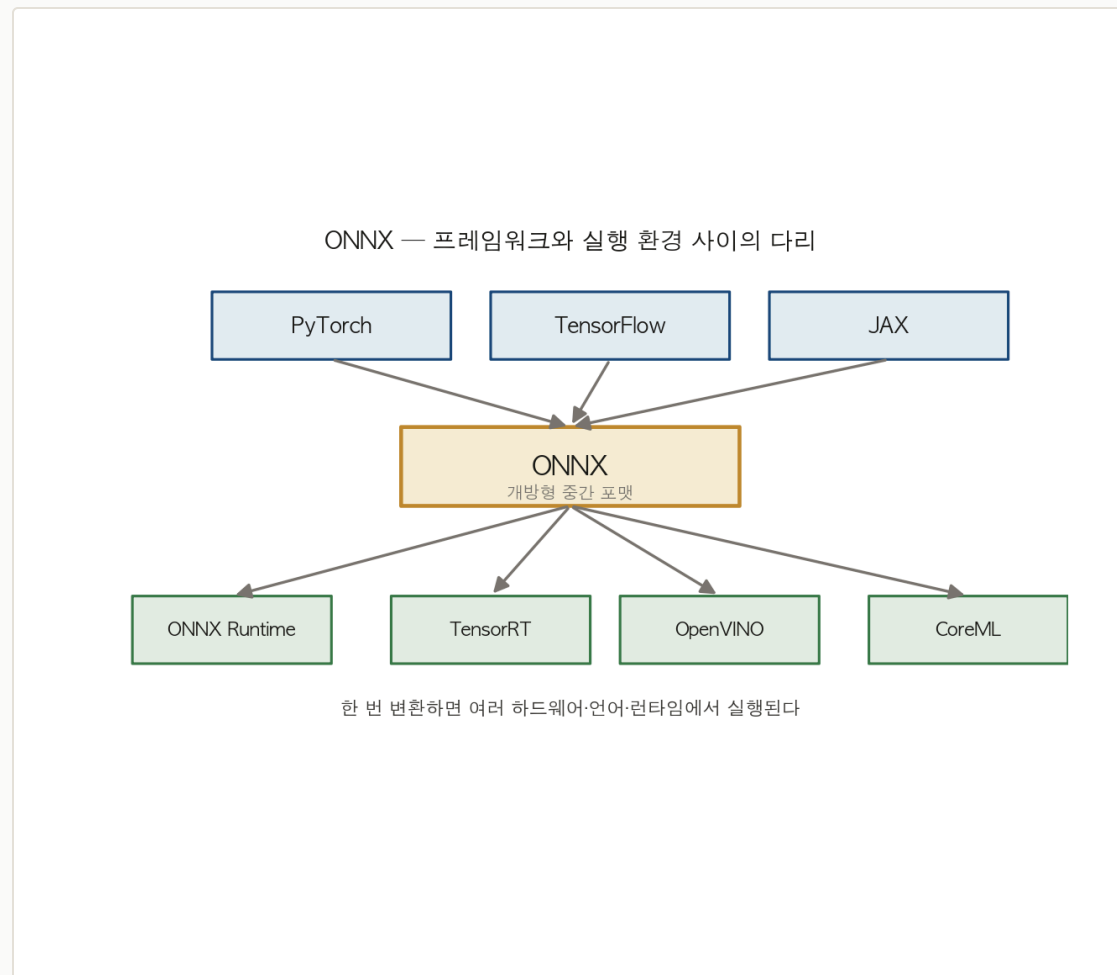
강점

그래프 최적화·연산 융합으로 추론이 빨라지고, TensorRT·OpenVINO 같은 가속 엔진과 붙습니다. 속도가 곧 비용인 프로덕션에 강합니다.

주의점

변환 시 `dynamic_axes` 로 배치 축을 열어 줘야 하고, PyTorch와 `1e-5` 수준 오차는 정상이니 `allclose` 로 확인합니다.

한 번 변환하면, 여러 곳에서 실행된다



프레임워크 → ONNX(중간 포맷) → 다양한 런타임·하드웨어.

01 위 — 어디서 왔나

PyTorch·TensorFlow·JAX 등 무엇으로 학습했든 ONNX 형식으로 내보낼 수 있습니다.

02 가운데 — 공통 포맷

ONNX는 모델을 표준 그래프로 표현한 **개방형 중간 포맷**입니다. 특정 프레임워크에 묶이지 않습니다.

03 아래 — 어디서 도나

ONNX Runtime·TensorRT·OpenVINO·CoreML 등 다양한 엔진에서 실행됩니다. C++·C#·모바일까지.

04 그래서 강점

학습 환경과 실행 환경을 **분리**합니다. 추론 속도·하드웨어 가속이 중요할 때 진가를 발휘합니다.

torch.jit.trace 와 script, 언제 무엇을

TorchScript로 바꾸는 두 방식 — 모델의 제어 흐름이 선택을 가릅니다.

예제 입력으로 기록

torch.jit.trace

동작 방식

예제 입력을 한 번 흘려보내, 실제로 거친 연산 경로를 그대로 기록합니다.

강점

대부분의 모델에 잘 맞고 안정적입니다. `if/else` 분기가 없다면 이 방식이 기본입니다.

한계

입력에 따라 경로가 갈리는 모델은, 기록된 한 경로만 남아 다른 입력에서 틀어질 수 있습니다.

코드를 분석

torch.jit.script

동작 방식

파이썬 코드를 분석해 제어 흐름(`if` · `for`)까지 TorchScript로 변환합니다.

강점

입력에 따라 다른 연산을 타는 모델도 정확히 담아냅니다.

한계

지원하지 않는 파이썬 문법이 있어 변환이 까다로울 수 있습니다. 이 과정은 `trace`를 기본으로 씁니다.

왜 .pth를 함부로 열면 안 되는가

state_dict의 `.pth`는 Pickle 기반입니다 — 편하지만 보안 위험이 따릅니다.

Pickle 기반

.pth · torch.save

임의 코드 실행 위험

Pickle 역직렬화는 파일에 심긴 파이썬 코드를 그대로 실행할 수 있습니다. 신뢰할 수 없는

`.pth`는 악성 코드의 통로가 됩니다.

그래서 weights_only

PyTorch 2.6+는 `torch.load`의 기본을 `weights_only=True`로 바꿔, 텐서만 안전하게 불러옵니다.

생태계에 묶임

포맷이 파이썬·파이토치에 종속적이고, 사람이 열어 읽기도 어렵습니다.

현대적 대안

safetensors

코드 실행이 불가능

순수 텐서 데이터만 담아, 파일을 열어도 코드가 실행될 여지가 없습니다. Hugging Face가 표준으로 채택했고, 이제 **PyTorch Foundation 공식 프로젝트**입니다.

빠르고 안전한 로딩

메모리 매핑으로 큰 모델도 빠르게 적재하고, 필요한 텐서만 부분 로딩할 수 있습니다.

받아 쓸 때의 선택

Day 1은 state_dict로 충분하지만, 공개 모델을 내려받을 땐 `.safetensors`가 보이면 우선 그쪽을 택하세요.

상황이 포맷을 결정한다

정답은 하나가 아닙니다. 목적에 따라 확장자가 달라집니다.

.pth

학습·실험 중

state_dict — 가장 간단하고 학습 재개에 적합. 이 과정
프로젝트의 **기본값**입니다.

.pt

Python 서빙

TorchScript — 클래스 정의 없이 로드. FastAPI 환경이면
state_dict도 무방합니다.

.onnx

성능 최적화

ONNX + ONNX Runtime — 추론 속도가 중요한 프로덕션.
하드웨어 가속 환경.

학습한 모델을 저장하고 다시 불러오기

오늘 배운 모든 내용을 하나의 흐름으로 엮습니다.

STEP 01

학습

MNIST로

`SimpleClassifier`를

학습합니다. 핵심은 학습이 아니라

학습 이후의 과정입니다.



STEP 02

저장

`model.eval()` 후 세 방식

(`state_dict·TorchScript·ONNX`)

으로 저장합니다. `eval`

호출은 필수입니다.



STEP 03

불러오기

각 파일을 **독립적으로** 다시

로드합니다. `state_dict`는

클래스 정의가, 나머지는 파일만

있으면 됩니다.



STEP 04

검증

원본과 추론 결과가 같은지

비교합니다. 저장은 됐는데 결과가

다르면 배포 후 오답을 내게 됩니다.



STEP 05

함수 분리

추론 로직을 `predict()` 함수로

떼어 둡니다. 다음 시간 FastAPI에

그대로 연결됩니다.

예제 모델과, 떼어 둔 추론 함수

직렬화 실습용 모델과, 이후 API에 그대로 붙일 `predict()` 를 미리 분리해 둡니다.

```
import torch.nn as nn

class SimpleClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 128), nn.ReLU(),
            nn.Linear(128, 10))
    def forward(self, x):
        return self.fc(x)

# 추론 로직을 함수로 분리 — 다음 시간 API가 이걸 부른다
def predict(model, x):
    model.eval()
    with torch.no_grad():
        return model(x).argmax(dim=1)
```

01 왜 클래스가 필요한가

state_dict는 가중치만 담으므로, 불러올 땐 이 `SimpleClassifier` 정의가 반드시 있어야 합니다.

02 forward()

입력이 어떤 연산을 거치는지 정의합니다. trace·ONNX 변환도 이 경로를 따라 그래프를 그립니다.

03 predict() 분리

전처리·추론을 한 함수로 묶어 두면, 다음 시간 FastAPI 엔드포인트가 그대로 호출합니다.

04 eval · no_grad

추론에선 추론 모드여야(`eval`) 하고, 기울기 계산이 필요 없습니다(`no_grad`).

MNIST로 학습하고 세 방식으로 저장한다



MNIST 손글씨 숫자 — 28×28 흑백 이미지를 10개 클래스로 분류한다.

01 데이터

0~9 손글씨 28×28 이미지. 이미 익숙한 분류 문제이므로 학습 코드 설명은 최소화하고 배포 흐름에 집중합니다.

02 저장 전 eval()

저장 직전 반드시 `model.eval()`. Dropout·BatchNorm은 학습 모드와 추론 모드 동작이 달라, 추론용은 eval에서 저장합니다.

03 weights_only=True

PyTorch 2.6+ 에서 `torch.load` 는 가중치만 안전하게 로드하는 `weights_only=True` 가 권장됩니다(Pickle 보안).

04 dynamic_axes

ONNX export 시 배치 축을 가변으로 지정해야 배포에서 다양한 배치 크기를 받을 수 있습니다.

세 방식으로 저장하고, 같은 결과인지 검증한다

`model.eval()` 후 저장하고, 각각 **독립적으로** 불러와 원본 출력과 비교합니다.

```
model.eval()                                # 저장 전 필수

# ① state_dict (가중치만)
torch.save(model.state_dict(), "models/model.pth")

# ② TorchScript (구조 포함)
ts = torch.jit.trace(model, example_input)
ts.save("models/model.pt")

# ③ ONNX (프레임워크 독립)
torch.onnx.export(model, example_input, "models/model.onnx",
                  dynamic_axes={"input": {0: "batch"}})

# 검증 - 원본과 같은 예측인가?
assert torch.allclose(original_out, loaded_out, atol=1e-5)
```

01 eval() 먼저

Dropout·BatchNorm이 추론 모드로 바뀝니다. 빠뜨리면 저장된 모델이 **다른 예측**을 냅니다.

02 example_input

trace와 ONNX는 예시 입력으로 그래프를 그립니다. 실제 입력과 같은 shape여야 합니다.

03 dynamic_axes

배치 축을 가변으로 열어, 배포에서 다양한 배치 크기를 받게 합니다.

04 allclose 검증

부동소수점 오차(`atol=1e-5`)는 허용하되, 세 방식 모두 원본과 일치해야 통과입니다.

저장-불러오기-추론 결과의 일관성 검증

배포 전 필수 절차 — 저장한 모델이 원본과 **같은 예측**을 내는지 반드시 확인합니다.

세 방식이 같은 답을 내는가. 세 파일을 각각 독립적으로 불러와 같은 입력으로 추론하고 원본과 비교합니다. 저장은 됐는데 결과가 다르면, 배포 후 **조용히 틀린 예측**을 내놓게 됩니다.

미세 오차는 정상입니다. ONNX Runtime은 부동소수점 연산 순서 차이로 살짝 어긋날 수 있습니다. `atol=1e-5` 수준은 정상으로 보고, `torch.allclose` 로 허용 범위 안인지 확인합니다.

배치 추론도 확인합니다. 실제 배포에선 한 번에 여러 입력이 들어옵니다. 단건과 배치가 같은 코드 경로를 타도록 설계하면, 트래픽이 늘어도 같은 함수가 버팁니다.

검증을 자동화하면 더 든든합니다. 새로 학습할 때마다 **저장 → 로드 → 일치** 검사가 자동으로 돌면, 배포 전에 직렬화 사고를 먼저 잡아냅니다.

추론 함수로 분리해 둡니다. `predict()` 로 떼어 두면 다음 시간 FastAPI가 그대로 호출합니다. 오늘의 산출물은 **검증된 모델 파일 + 재사용 가능한 추론 함수**입니다.

요청 하나가 서버 전체를 멈춘다면

Day 3에서 깊이 다룹니다 — 여기선 왜 문제인지 감만 잡습니다.

동기 (blocking)

한 번에 하나씩

줄을 세운다

추론 한 건이 2초 걸리면, 그 2초 동안 도착한 모든 요청이 순서를 기다립니다. 동시 사용자가 늘수록 대기는 눈덩이처럼 커집니다.

헬스체크도 막힌다

로드밸런서가 "살아있니?"라고 물어도 막힌 서버는 답을 못 해, 멀쩡한 서버까지 죽은 줄 알고 뺄어버릴 수 있습니다.

비동기 (async)

대기 시간을 활용

기다리는 동안 다른 일

DB·외부 API·파일 같은 **I/O를 기다리는 사이**, 이벤트 루프가 다른 요청을 처리합니다.

그래도 만능은 아니다

CPU를 길게 쓰는 모델 추론은 `async` 만으로 부족합니다. `run_in_executor` 로 따로 빼야 진짜 효과 — Day 3의 핵심입니다.

8일 동안 무엇을 만드나

오늘은 그 첫걸음입니다. 배경지식이 없어도, 이 지도를 따라가면 됩니다.

STEP 01

Day 1 — 기반 (오늘)

직렬화·REST·환경. 모델을 파일로
굳히고 API의 언어를 익힙니다.



STEP 02

Day 2~3 — API 본체

FastAPI로 추론 엔드포인트를
만들고, 비동기·에러 처리로
튼튼하게 만듭니다.



STEP 03

Day 4 — 화면

Streamlit으로 사용자가 직접 쓰는
대시보드를 붙이고, 프론트-백엔드
구조를 익힙니다.



STEP 04

Day 5~7 — 실전

정형 데이터 예측(프로젝트
1) · 인증과 이미지 처리 ·
트랜스포머 챗봇(프로젝트 2)을
차례로 서빙합니다.



STEP 05

Day 8 — 내 서비스

배운 것을 모아 나만의 모델 서빙
서비스를 처음부터 끝까지 완성합니다.

오늘 닫고, 다음을 연다

오늘 한 일

- ▶ 학습과 배포의 차이, "내 컴퓨터에서는 되는데"의 원인을 규명
- ▶ `venv` · `requirements.txt` 로 실행 환경을 통제·재현
- ▶ REST 원칙·**GET/POST**·상태 코드·**JSON** 으로 API의 언어 습득
- ▶ **state_dict**·**TorchScript**·**ONNX** 세 직렬화 방식과 선택 기준
- ▶ MNIST 모델을 저장→불러오기→**검증**하고 추론 함수로 분리

다음 — DAY 2

- ▶ 오늘 만든 **추론 함수**를 FastAPI 엔드포인트로 감싸기
- ▶ **Pydantic** 스키마로 입력 검증과 자동 문서화
- ▶ **Swagger UI** 로 API를 눈으로 테스트
- ▶ Path·Query·Body 파라미터로 요청 설계하기
- ▶ 모델 로딩을 서버 시작 시 **한 번만** — 성능의 첫 단추

완벽한 모델보다 동작하는 서비스가 먼저다. 오늘 검증한 모델 파일과 추론 함수가, 내일 API의 심장이 된다.